

**İZMİR UNIVERSITY OF ECONOMICS
FACULTY OF ENGINEERING
SOFTWARE ENGINEERING**

FENG 498 PROJECT REPORT



Gamification in Teaching Software Design Patterns

Author(s):

Gizem Ulusoy 20200601055

Seray Cebeci 20200601016

Supervisor:

Dr. Öğr. Üyesi Ufuk Çelikkan

OUTLINE

Title page

Table of Contents

1. Abstract

2. Introduction

3. Literature Review

4. Methodology

4.1. Planning and Pattern Scenario Design

4.1.1. Adapter Pattern

4.1.2. Command Pattern

4.1.3. Facade Pattern

4.1.4. Iterator Pattern

4.1.5. Observer Pattern

4.1.6. Singleton Pattern

4.1.7. Template Pattern

4.1.8. Abstract Factory Pattern

4.2. Development Progress and Implementation Samples

4.3. Tools and Technologies Used

5. Results and Discussion

6. Conclusion

7. References

1. Abstract

In our project, we designed and developed an educational game that aims to teach object-oriented design patterns through interactive gameplay and real-life analogies. Each level introduces a specific design pattern and challenges the player to apply it within a familiar context. The goal is to help learners understand not only the structure of each pattern, but also *when* and *why* to use it in software development.

The motivation behind the project stems from our own experiences as software engineering students. We noticed that design patterns, while essential for writing clean and maintainable code, are often taught in abstract and theory-heavy ways that are hard to relate to actual coding problems. For beginners, especially those new to object-oriented programming, understanding these patterns can be confusing without hands-on examples. This inspired us to build a tool that bridges the gap between theory and practice using an engaging, game-based format.

To develop the game, we used the Unity engine and implemented a level-based progression system. Each level simulates a real-life scenario where a design pattern naturally fits — such as a family using the Singleton pattern to manage a shared resource, or a restaurant menu system illustrating the Strategy pattern. The visual and interactive format was designed to help players internalize each concept more effectively than with traditional lectures or textbooks.

Although we were unable to conduct formal user testing due to time constraints, we played through the game ourselves and received informal feedback from peers. These initial reactions were positive, suggesting that the approach has potential to make abstract programming ideas more understandable and engaging. However, more structured evaluations would be needed to measure learning outcomes objectively.

In conclusion, this project demonstrates how gamification can be used to teach complex technical concepts in a more intuitive and memorable way. Our game serves as an early prototype for a larger vision — creating educational tools that make programming principles more accessible, especially for beginners. With further refinement and testing, this method could contribute meaningfully to computer science education.

2. Introduction

In software development, design patterns provide well-established solutions to common problems in software architecture and code organization. These patterns offer a shared language for developers and promote consistency and maintainability. However, for many students and beginners in object-oriented programming (OOP), design patterns can feel overly abstract and disconnected from real-world programming challenges. Understanding their purpose and implementation often requires a cognitive leap that traditional lectures and diagrams alone may not fully provide.

To address this learning gap, we developed an educational game that introduces and teaches design patterns through interactive gameplay, visual representation, and real-life analogies. Each level in the game is dedicated to a specific design pattern and presents players with tasks that require applying that pattern to solve a contextual problem. For example, one level might demonstrate the Singleton pattern through a household analogy where a shared object must be controlled, while another level uses a shopping scenario to illustrate the Strategy pattern. These scenarios are intentionally familiar to help players internalize the logic behind each pattern.

Our goal was to make the learning process more intuitive, engaging, and effective by allowing players to actively apply the concepts rather than passively read or memorize them. We used Unity to build the game and implemented level mechanics that blend theoretical explanations with hands-on experiences. The game also provides visual breakdowns of each pattern's structure (such as participants, roles, and responsibilities), reinforcing comprehension through repetition and practice.

Due to time constraints, we did not conduct a full-scale user testing process. However, we extensively tested the game ourselves and received informal feedback from peers who interacted with early builds. While we lack quantifiable data, the general response indicated that the game format helps demystify design patterns and supports a more practical understanding of their use in programming.

In conclusion, this project presents a prototype of a game-based learning tool that simplifies and enhances the understanding of design patterns. It demonstrates that abstract software concepts can be made more tangible and enjoyable when embedded in meaningful, interactive experiences. Future improvements include expanding pattern coverage, refining feedback systems, and conducting formal evaluations to measure learning impact.

2.1. Problem Statement

Design patterns are fundamental tools in software development, enabling developers to create structured, consistent, and scalable systems. However, for students and beginners—especially those new to object-oriented programming—these patterns often seem abstract, theoretical, and disconnected from real-world applications. Traditional teaching methods, which rely heavily on static diagrams and technical jargon, can limit comprehension, reduce engagement, and ultimately hinder the learner’s ability to apply these concepts in practice. This challenge highlights a broader educational gap: the need for more inclusive and accessible teaching methods in programming education. To address this, our project introduces an educational game that simplifies the learning of design patterns through interactive scenarios and real-life analogies. By making abstract programming concepts more tangible, the game supports **UN Sustainable Development Goal 4: Quality Education**, which aims to ensure inclusive, equitable, and effective learning opportunities for all—particularly in areas like digital literacy and technical skill development.

2.2. Motivation

Design patterns are among the most critical building blocks in software engineering, offering standardized solutions to frequent design challenges. They promote code clarity, reusability, and long-term maintainability—skills essential not only in academic contexts but also in professional software development. Despite their value, these patterns are often taught in ways that fail to capture their practical utility. Diagrams and textbook definitions may explain structure, but they frequently lack context, making it difficult for learners—especially beginners in object-oriented programming—to internalize the patterns or understand when and how to use them. This creates a disconnect between what students learn and what they can apply, which ultimately weakens their confidence and problem-solving abilities.

Our project is motivated by the need to close this educational gap through a more engaging and experiential approach. By turning the learning process into a game, we provide students with an opportunity to interact with design patterns in dynamic, meaningful scenarios. The use of real-life analogies allows abstract software principles to take on familiar forms, making them easier to understand, relate to, and remember. This not only enhances conceptual clarity but also encourages active learning—an approach shown to improve long-term retention and transfer of knowledge. Beyond technical education, this project contributes to the broader goal of developing accessible, inclusive learning tools in digital fields, empowering students with the foundational thinking required in modern software development.

3. Literature Review

One of the common challenges in software engineering education is helping students grasp abstract concepts like design patterns in a lasting and practical way. Traditional teaching methods often fall short in creating an interactive and engaging environment that shows how these patterns are used in real-world scenarios. This project aims to improve the teaching process of design patterns by applying gamification techniques. Game elements such as points, badges, level progression, and leaderboards will be used to increase students' motivation and encourage active participation.

Previous research shows that gamification can positively affect learning outcomes. According to Furdu et al. [1], it increases classroom engagement and student motivation. Similarly, Stott and Neustaedter [2] highlight that competitive and interactive environments created through gamification help students stay focused and motivated. Gamification is not just a way to make learning fun — it can also serve as a valuable educational tool that supports feedback, progress tracking, and goal-oriented behavior.

Studies focused on programming and software-related subjects suggest that gamification is particularly effective in these areas. Aibar-Almazán et al. [3] found that using tools like Kahoot! improved students' motivation and understanding. Sailer & Sailer [4] showed that combining game elements with flipped classroom methods helped students come to class better prepared and more engaged. Wiggins [5] also noted that gamification strengthened students' connection to the course beyond just academic performance.

More specifically, gamification has been shown to improve outcomes in programming courses. Maryono et al. [6] reported that it boosted confidence, participation, and success in technical subjects. Elshiekh & Butgerit [7] emphasized that students were more eager to engage with problem-solving when presented in a game-like environment. Deterding et al. [8] also offered a solid theoretical background for gamification, outlining how game design principles can be meaningfully applied to learning.

This project will explore how gamification strategies can be tailored to teaching design patterns. Students will be given tasks such as identifying patterns, analyzing code, or developing simple application scenarios. These tasks will be supported by motivational elements like points or rewards. By observing how students respond, we aim to better understand the educational value of gamification in software engineering and gather data that may help guide future instructional approaches.

4. Methodology

4.1. Planning and Pattern Scenario Design

Our first step in developing the project was to find a way to make software design patterns—usually seen as abstract and difficult—more understandable and relatable for beginners. Rather than jumping into technical development, we focused on creating meaningful, story-driven scenarios that represent each pattern in real-life situations. This method was especially important because our main goal was not just to *tell* what a pattern is, but to help learners *feel* how it works through interaction.

We believed that using everyday examples could bridge the gap between theory and practice. To achieve this, we held brainstorming sessions where we discussed how each design pattern could appear in ordinary settings such as a home, a toy store, or a workplace. These imaginary situations were then written as short scenarios, which later shaped the flow and logic of each game level. The scenario-based approach also helped us make sure that each pattern wasn't only described but experienced through actions and decisions in the game world.

4.1.1 Adapter Pattern

To teach the Adapter Pattern, we designed several real-life-inspired level concepts where the player connects incompatible systems using the correct adapters. The core idea across all levels is to help the player understand how an adapter allows one interface to work with another without changing their original structures.

One scenario is based on international travel, where the player acts as a traveler who must use the correct power plug adapter to charge devices in different countries. Each country has a unique socket type, and the player needs to match the correct adapter to avoid device failure. As the level progresses, more devices and plug types are introduced, demonstrating how a single adapter can support different systems or how some systems may require unique adapters.

Another level uses LEGO blocks, where the player builds structures using bricks from different sets that don't directly fit together. By placing special "adapter bricks" between incompatible pieces, the player creates bridges, buildings, and machines. This clearly visualizes how adapters allow systems with different formats to connect.

A third scenario involves language translation, where the player takes the role of a translator in a global meeting. Each character speaks a different language and cannot understand the others directly. The player must assign the correct translator to facilitate communication. Choosing the

wrong adapter results in miscommunication, adding a fun challenge while reinforcing the pattern's purpose: enabling communication between incompatible interfaces.

4.1.2 Command Pattern

In this level, the Command Pattern is taught through a TV remote control simulation. The player controls a TV by pressing buttons on a remote, with each button mapped to a specific command, such as turning on the TV, changing the channel, or increasing the volume. The TV itself performs the requested operations, while the human character interacts with the remote.

The level begins with a simple task: turning on the TV. When the player presses the power button, a "TurnOnCommand" is sent from the remote to the TV, and the screen updates with the message "TV is ON." Players can also switch channels using channel commands and increase volume using volume commands, each resulting in visible feedback on the screen. If the player attempts to turn on the TV when it is already on, the system detects this and prevents the redundant action—showing that command state management is possible.

To demonstrate the flexibility of the pattern, the remote includes empty buttons that players can assign custom actions to, such as jumping directly to "Channel C." This introduces the idea of extensibility, where new commands can be added without altering existing code. Additionally, the remote includes a command sequence feature: when activated, the player can press multiple command buttons in order, and those commands will be executed automatically at intervals, simulating a timed macro sequence.

The level also supports an Undo function. For example, after changing the channel, pressing the Undo button returns the TV to the previous channel, teaching command history and reversibility. Finally, an optional feature allows the system to record the command history, showcasing the benefit of using Command Pattern to track user actions.

This level illustrates how the Command Pattern separates the sender from the receiver, adds flexibility in defining new commands, and enables powerful extensions like undo, redo, logging, and delayed execution.

4.1.3 Facade Pattern

This level demonstrates the Facade Pattern through a smart home scenario. The main character, Ayça, arrives home after a tiring day at work. Instead of turning on each device individually, she uses a single button in her smart home mobile app labeled "I'm Home" to activate multiple systems at once. This interaction represents the Facade Pattern, where a unified interface

simplifies the control of complex subsystems.

At the beginning of the level, Ayça walks to the front door with a tired expression. She opens the smart home app on her phone and presses the “I’m Home” button. In response, the smart home system turns on the air conditioner, lights, television, and plays relaxing background music. Once everything is active, Ayça smiles and comments, “I was too tired to do all these one by one... doing it all with one button made it so much easier.”

In this scenario, Ayça is the client who interacts with a central control system, the smart home controller acts as the facade that manages all subsystems, and the individual devices such as the lights, TV, music player, and air conditioner operate as the subsystems. This level helps players understand how the Facade Pattern simplifies interactions with complex systems by providing a single access point to multiple related components. Instead of dealing with each system separately, the player controls everything through one clean interface, highlighting the real-world benefit of decoupling complex subsystems behind a simplified façade.

4.1.4 Iterator Pattern

This level teaches the Iterator Pattern through a warehouse and supermarket scenario. The player observes how a store manager delegates the task of transferring items from a warehouse—without accessing its internal structure—by using specific helper objects that follow the iterator logic.

The scene starts with a supermarket manager asking for pasta from the warehouse. However, there is a wall separating the store from the warehouse, and the manager cannot access or even see what's inside. Instead, they rely on a set of helper characters that each perform one part of the iteration process, simulating the behavior of an Iterator interface with methods like `first()`, `next()`, `isDone()`, and `currentItem()`.

The `first()` character enters the warehouse and retrieves the first box of pasta, placing it on the store shelf. The manager then asks `isDone()` to check whether the task is finished. `isDone()` returns and reports that more items remain. The `next()` character is then sent in to locate the next item's position, and it informs `currentItem()`, who picks up the pasta and delivers it to the shelf. This process repeats one more time, following the same steps: `isDone()` checks again and finally reports that all pasta boxes have been retrieved.

At the end of the level, the manager and all helper characters celebrate the successful completion of the task, with visual effects indicating satisfaction. This level clearly demonstrates how the Iterator Pattern allows traversal of a collection without exposing its internal structure, and

how each method works in coordination to complete the process.

4.1.5 Observer Pattern

This level explores the Observer Pattern through three interconnected scenarios based on student Ahmet's experiences. It demonstrates how a change in one object can notify and update multiple dependent objects, and how this pattern can be extended with pull mechanisms or combined with a mediator.

The main scene starts with Ahmet sitting at his desk and checking his exam result on a computer. The screen displays: "Your teacher has entered your grade. Grade: 90." After seeing his grade, Ahmet leaves the room and enters a new space where his mother, teacher, and school principal are waiting. Ahmet tells each of them his grade, and they all react differently. His mother offers to buy him a bicycle, the teacher promises a gold star, and the principal announces a reward certificate. In this setup, Ahmet is the subject whose state change triggers reactions in the other characters.

In another version, the pull model is introduced. Ahmet checks both his university exam score and the university he has been accepted into. Later, his mother enters and asks which university he got into, and his teacher later asks for his ranking. In this model, the observers pull the data themselves instead of being directly pushed, which highlights an alternative approach within the Observer Pattern.

The third version shows how the Observer Pattern can work alongside the Mediator Pattern. Ahmet is caught cheating during an exam. The Exam Proctor notifies Ahmet's teacher, who then informs Ahmet's mother and the principal. The mother becomes upset, and the principal initiates a disciplinary process. Here, the teacher acts as a mediator between the event and the observers, distributing notifications and triggering their reactions.

Across all three scenarios, the level uses animations, speech bubbles, and character expressions to make the message delivery and reactions clear. This level effectively teaches the Observer Pattern's versatility: it can be implemented directly, via pull logic, or through a central mediator to distribute notifications to multiple observers.

4.1.6 Singleton Pattern

This level teaches the Singleton Pattern through a school-based scenario, where multiple students with different needs all rely on a single person to handle their requests—the school principal. The scene begins in a classroom where three students, Gizem, Seray, and Taylan, are

discussing their concerns. Gizem wants to apply for a scholarship, Taylan is trying to change one of his courses, and Seray needs a student certificate. However, none of them knows whom to talk to about their issue.

As they prepare to leave the classroom and search for help, their teacher Fatma stops them and explains that there is one person in the school who handles all such requests: the principal, Mr. Mehmet. She tells them that going to different people would cause confusion and inconsistency, but having one central authority ensures everything is handled in an organized way. The students understand and head to the principal's office.

Inside the principal's room, each student expresses their request, and the principal listens carefully. He assures them that they are in the right place and that he is the one responsible for managing all such operations. This interaction represents the core idea of the Singleton Pattern—there is only one object responsible for handling all related tasks. The students act as clients who access this single instance to get their requests fulfilled. The teacher, who guided them, serves as the access point through which clients are directed to the Singleton.

By the end of the scene, it becomes clear that no matter how different the students' needs are, they are all handled consistently and centrally by one authority figure. This reinforces the purpose of the Singleton Pattern: ensuring controlled access to a single, shared instance in a system where consistency and coordination are essential.

4.1.7 Template Pattern

This level introduces the **Template Method Pattern** through a series of real-life-inspired scenarios, where a fixed sequence of steps is followed, but specific parts of the process are left customizable. The objective is to help players understand how a base algorithm can define a skeleton of operations, while allowing certain steps to be overridden depending on context.

The first scenario is set in a travel agency in İzmir, where a staff member named Ayça manages tour packages. On the same day, she receives two customer requests: one for a tour of Istanbul and one for a tour of Cappadocia. Both tours follow the same structure: meeting point, transportation, sightseeing, meal, and farewell. The system Ayça uses applies a fixed template for every tour. The meeting point is always in Alsancak, transportation is by bus, the meeting time is 7 AM, and the farewell point is always Balçova. However, the sightseeing and meal steps vary based on the destination. For the Istanbul tour, Ayça fills in “visiting Topkapi Palace and Hagia Sophia” and “lunch at Sultanahmet Köfteci.” For the Cappadocia tour, she selects “hot air balloon ride” and “testi kebab” for the customizable steps. Thanks to this structure, Ayça doesn't have to

build a new schedule from scratch for each tour—she just updates the flexible parts.

In the second scenario, the same principle is applied to a cooking app. Each recipe follows the same steps: ingredient preparation, cooking, and presentation. While preparation and presentation remain consistent across recipes, the cooking step varies. For instance, soup recipes use boiling, whereas cake recipes involve baking. The player learns how a standard process can remain intact, while allowing variation where necessary.

The third example is based on online shopping. A customer orders products from an e-commerce platform, which always follows the same sequence: add to cart, make payment, apply security verification, ship the item, deliver, and offer a return option. However, the shipping and return policies differ depending on the product type. For example, electronic devices require special packaging and have shorter return windows, while clothing items use standard packaging and offer longer return periods. Through this scenario, players see how template methods provide structure and consistency while allowing necessary flexibility for specific situations.

Each of these cases reinforces the idea that the Template Method Pattern ensures a standard flow of operations while supporting limited customization in predefined areas. The player interacts with systems that demonstrate both fixed and variable logic, gaining insight into how abstract methods can be implemented differently without altering the overall algorithm.

4.1.8 Abstract Factory Pattern

This level introduces the **Abstract Factory Pattern** through a scenario inspired by assembling furniture from Ikea. The scene begins with two different wardrobe models that need to be assembled using matching parts. At first, the player tries to build both wardrobes using mixed components from different collections, which leads to incorrect fits, missing connections, and visual mismatches.

To solve this problem, the player activates the abstract factory system, which provides a consistent set of components specifically designed for each wardrobe type. For example, when assembling Wardrobe A, the player receives shelves, doors, screws, and handles from Factory A—all of which fit together perfectly and follow the same design language. When building Wardrobe B, all parts come from Factory B, ensuring a different but equally cohesive result. Each factory produces a family of related objects that are meant to work together seamlessly.

Boxes are labeled by type, and the player selects the correct box to extract the right set of parts. This reinforces the concept that instead of choosing components individually and risking mismatches, the abstract factory provides grouped products that ensure compatibility. Through this

process, the player learns how abstract factories manage the creation of related objects without specifying their concrete classes, making the system flexible and scalable for multiple furniture models.

By the end of the level, both wardrobes are successfully assembled using parts from their respective factories. The player sees firsthand how abstract factories help ensure consistency and compatibility across product families while simplifying the construction process.

4.2. Development Progress and Implementation Samples

As of the current development phase, the Abstract Factory Pattern level has been fully implemented and is functional within the game. The level successfully demonstrates how different component sets (e.g., for different wardrobes) are managed by separate factories and assembled using cohesive parts. The interaction is visual, guided, and reinforces the concept of consistency across product families.

In addition, the Iterator Pattern level is near completion. The gameplay flow—where helper characters traverse through a warehouse to retrieve items sequentially—is mostly implemented. Remaining tasks include finalizing animations and refining feedback effects to clearly visualize each iteration step.

Below are screenshots showcasing the in-game implementation of these pattern-based levels.

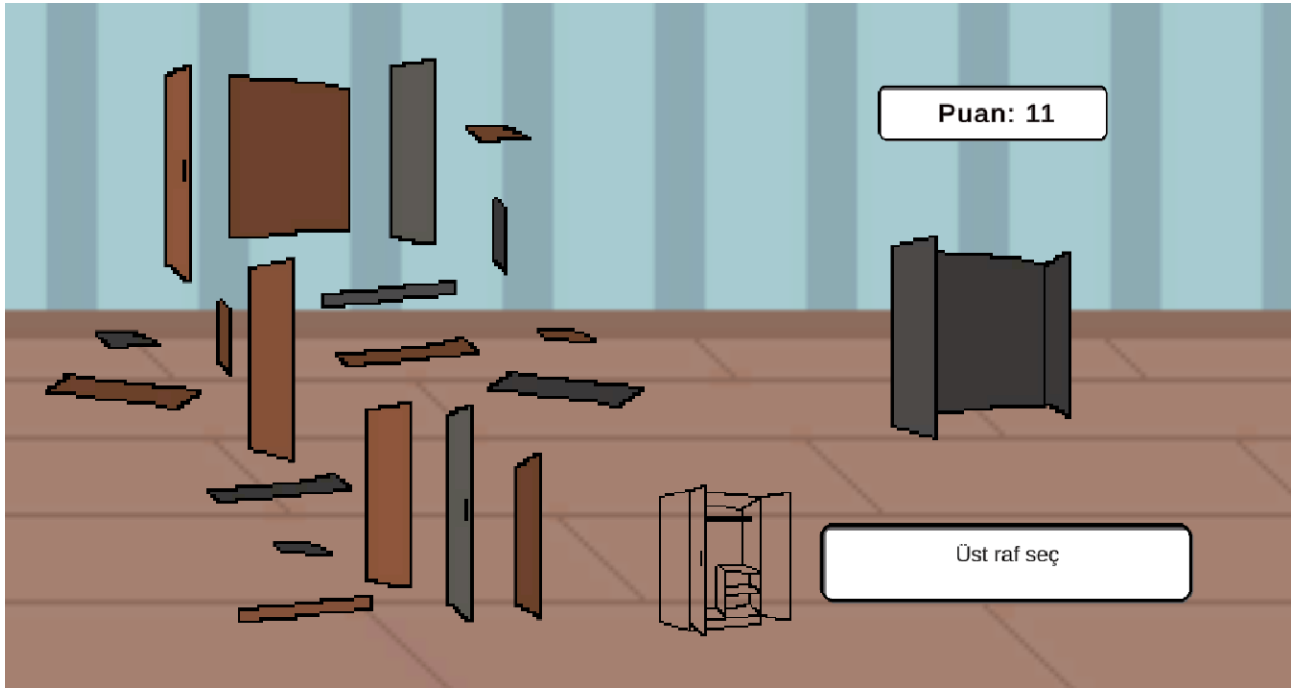


Figure 1. Abstract Factory level: assembling compatible furniture components from separate factories.

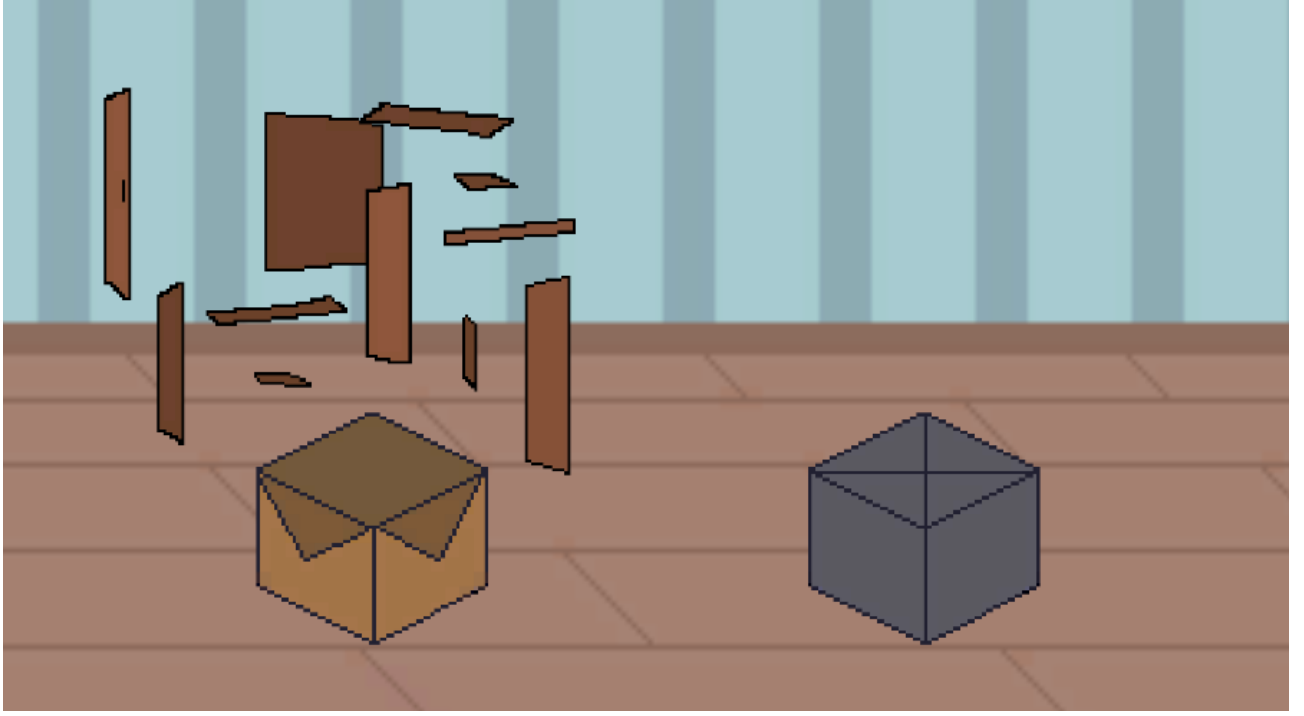


Figure 2. *Abstract Factory level showing parts from different factory boxes, each forming a compatible wardrobe*



Figure 3. *A snapshot from the Iterator Pattern level where each helper character performs a specific step—first(), next(), isDone(), and currentItem()—to move items from the warehouse to the store in a controlled sequence.*

4.3. Tools and Technologies Used

The project was developed using **Unity** as the main game engine, with **C#** as the programming language. We used **JetBrains Rider** as our primary development environment for scripting. For online collaboration and code sharing, we worked through **GitHub**. Pixel art assets were created and

animated using **Aseprite**, which allowed us to produce visually consistent and lightweight game graphics. To collaborate on written content such as design documents and scenario scripts, we used **Google Docs**, ensuring real-time access and feedback within the team.

5. Results and Discussion

By the end of our development process, the level based on the Abstract Factory pattern was fully completed, while the Iterator pattern level reached its finalization stage. Both levels aim to convey the structure and logic of the patterns through visual cues and real-life analogies. In the Abstract Factory level, the way furniture parts emerge from designated factory boxes and combine seamlessly to create coherent wardrobes effectively reflects the core idea of the pattern. Similarly, the Iterator level illustrates the sequential traversal of a collection using iterator functions, helping users grasp each method's specific role through animated interactions.

After completing the initial levels, we conducted our own internal evaluation to assess how well the scenarios reflect the intended patterns. We found that explaining abstract concepts through familiar contexts—such as assembling furniture—significantly improved clarity. For example, showing that each wardrobe can only be assembled using parts from its own box directly visualized the essence of the Abstract Factory pattern.

Although the gameplay mechanics and visual flow remain at a basic level, each level maintains a progression that aligns structurally with the design pattern it represents. So far, no external user testing or formal feedback sessions have been conducted; this is a direction reserved for future improvement. Still, the project in its current state demonstrates how game-based learning methods can be applied to teaching software design patterns in an experiential way.

In conclusion, the developed levels suggest that associating software design patterns with everyday scenarios holds promising educational potential. Despite the limited scope and the lack of user testing at this stage, our findings indicate that an interactive and visual approach can contribute to a clearer understanding of abstract concepts. With further testing and content expansion, this method has the potential to become a supportive tool in computer science education.

6. Conclusions

In this project, our goal was to make software design patterns more understandable and intuitive by embedding them into an interactive, game-based learning environment. Instead of relying on traditional diagrams or textual explanations, we designed scenario-driven levels where each one visually and functionally represented a specific design pattern. These levels were inspired by real-life interactions and structured in a way that allowed players to experience how each pattern works within a practical context. For example, in the Abstract Factory level, assembling wardrobes using parts from matching factory boxes clearly demonstrated the idea of creating compatible product families. Similarly, in the Iterator level, retrieving items from a warehouse step by step helped visualize the concept of traversing a collection without knowing its internal structure.

Our internal evaluations suggest that this form of storytelling and simulation can simplify the learning of abstract programming concepts. In the Abstract Factory scenario, the clear rule that each wardrobe could only be assembled from its own box effectively illustrated the pattern's logic. The Iterator level, where each method was embodied by a unique character, also supported understanding through visual action.

At the heart of this approach is the use of relatable, narrative-based scenarios to reflect design principles. For instance, the Singleton pattern was portrayed through a school principal who manages all student requests, illustrating centralized control. The Command pattern took the form of a TV remote, showing how commands are sent and executed. The Facade pattern was illustrated with a smart home system where a single button controlled multiple devices, reinforcing the value of a simplified interface. These stories were crafted not just to describe the patterns but to help players feel how they operate in practice.

That said, there are some limitations to our approach. We have not yet conducted formal user testing or compared our method to traditional teaching tools. The levels remain visually simple, and their educational impact has not been quantitatively measured. In addition, more complex patterns may require advanced interactivity beyond what we currently implemented.

Future improvements could include structured usability studies, performance analysis, and expanding the visual and interactive elements of the game. More design patterns could be added to increase the scope of the experience, and collaboration with educators could help align the project with classroom teaching.

While still at an early stage, this project demonstrates the potential of combining storytelling, interaction, and game mechanics to teach software design principles. We believe this approach offers a promising direction for supporting computer science education through visual and experiential learning tools.

7. References

- [1] I. Furdu, C. Tomozei, and U. Köse, "Pros and Cons Gamification and Gaming in Classroom," arXiv preprint arXiv:1708.09337, 2017. [Online]. Available: <https://arxiv.org/pdf/1708.09337>
- [2] A. Stott and C. Neustaedter, "Analysis of Gamification in Education," Simon Fraser University, 2013. [Online]. Available: <https://carmster.com/clab/uploads/Site/Stott-Gamification.pdf>
- [3] A. Aibar-Almazán, Y. Castellote-Caballero, M. D. C. Carcelén-Fraile, Y. Rivas-Campo, and A. M. González-Martín, "Gamification in the classroom: Kahoot! as a tool for university teaching innovation," *Frontiers in Psychology*, vol. 15, 2024. [Online]. Available: <https://www.frontiersin.org/journals/psychology/articles/10.3389/fpsyg.2024.1370084/full>
- [4] M. Sailer and M. Sailer, "Gamification of in-class activities in flipped classroom lectures," *British Journal of Educational Technology*, vol. 52, no. 1, pp. 75–90, 2021. [Online]. Available: <https://bera-journals.onlinelibrary.wiley.com/doi/full/10.1111/bjet.12948>
- [5] B. L. Wiggins, "The Guided Classroom: Using Gamification to Engage and Motivate Undergraduates," *Journal of the Scholarship of Teaching and Learning*, vol. 16, no. 3, pp. 109–123, 2016. [Online]. Available: <https://scholarworks.iu.edu/journals/index.php/josotl/article/view/22119/29442>
- [6] D. Maryono, Budiyono, Sajidan, and M. Akhyar, "Implementation of Gamification in Programming Learning: Literature Review," *International Journal of Information and Education Technology*, vol. 12, no. 12, pp. 1057–1063, 2022. [Online]. Available: <https://www.ijiet.org/vol12/1771-ijiet-5754.pdf>
- [7] R. Elshiekh and L. Butgerit, "Using Gamification to Teach Students Programming Concepts," *Open Access Library Journal*, vol. 4, no. 8, pp. 1–7, 2017. [Online]. Available: <https://www.scirp.org/journal/paperinformation?paperid=78480>
- [8] S. Deterding, D. Dixon, R. Khaled, and L. Nacke, "From game design elements to gamefulness: Defining 'gamification'," in *Proc. 15th Int. Academic MindTrek Conf.*, 2011, pp. 9–15. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/8312377>
- [9] M. E. Ortiz Rojas, K. Chiluiza, and M. Valcke, "Gamification in computer programming: effects

on learning, engagement, self-efficacy and intrinsic motivation," in Proceedings of the 11th European Conference on Games Based Learning (ECGBL 2017), Graz, Austria, 2017, pp. 507–514. [Online]. Available: <http://hdl.handle.net/1854/LU-8607271>

